
SuperMXMai 一键部署方案

MxM研究部

2026年

目录

SuperMXMai 一键部署方案	1
一、当前架构梳理	2
1.1 现有服务	3
1.2 现有初始化脚本	4
1.3 现有环境变量	5
二、部署方案设计	6
2.1 设计原则	7
2.2 部署阶段划分	8
三、完整脚本实现	9
3.1 主入口：deploy.sh	10
deploy.sh — SuperMXMai 一键部署脚本	11
=====	12
配置	13
=====	14
颜色输出	15
创建日志目录	16
记录完整日志（同时输出到屏幕）	17
=====	18
参数解析	19
=====	20
=====	21
Phase 0: 环境检查	22
=====	23
=====	24
Phase 1: 基础设施 (Docker)	25
=====	26
=====	27
Phase 2: 数据库初始化	28
=====	29
=====	30
Phase 3: 账户初始化	31

=====	32
=====	33
Phase 4: 服务启动	34
=====	35
=====	36
Phase 5: 健康检查	37
=====	38
=====	39
主流程	40
=====	41
3.2 docker-compose.yml (新增)	42
docker-compose.yml — 基础设施服务	43
3.3 .env.example (新增)	44
=====	45
SuperMXMai 环境变量配置	46
=====	47
复制此文件为 .env 并填入实际值	48
---- 数据库 ----	49
Supabase 本地开发: postgres://postgres:postgres@localhost:5432/postgres	50
Supabase 云端: postgres://postgres.xxx.supabase.co:5432/postgres? sslmode=require	51
---- JWT ----	52
生成方式: node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"	53
---- 管理员账户 ----	54
---- 第三方 API ----	55
---- MinIO (本地开发) ----	56
---- R2 (可选, 生产用) ----	57
---- CORS ----	58
---- 代理 (可选) ----	59
3.4 Makefile (新增)	60
Makefile — 常用部署命令	61
默认目标: 完整部署	62
分阶段部署	63
停止所有服务	64
清理	65

查看日志	66
状态检查	67
四、部署流程图	68
五、使用方式	69
5.1 首次部署	70
1. 复制环境变量模板	71
编辑 .env 填入实际值	72
2. 一键部署	73
5.2 后续部署（增量）	74
只启动服务（数据库已初始化，跳过基础设施和数据库）	75
查看状态	76
查看日志	77
5.3 完整重置	78
清理所有数据，重新部署	79
六、当前问题与改进建议	80
6.1 现有问题	81
6.2 待完成项（建议 Claude Code 执行）	82

SuperMXMai 一键部署方案

日期：2026-04-28

一、当前架构梳理

1.1 现有服务

服务	端口	职责	依赖
Supabase	5432 (DB) + 8000 (PostgREST)	PostgreSQL + REST API	无
MinIO	9000	S3 存储 (task media)	无
mxmauth	4001	用户认证、JWT 签发	Supabase
mxmcgi	4003	AI 内容生成核心	Supabase, MinIO
mxmpay	4002	支付、钱包	Supabase
mxmagent	4004	Agent / Smartflow	Supabase
mxmnotify	4005	通知、SSE	Supabase
gateway	3000	统一入口、路由转发	mxmauth, mxmcgi, mxmpay, mxmagent, mxmnotify
web	5173 (dev)	前端	gateway

1.2 现有初始化脚本

```
init-project.ts    → 串行执行数据库初始化
├─ init-database   → mxmdata/init:db (schema SQL)
├─ create-admin-user → mxmdata/create:admin
└─ migrate-kb-defaults → mxmdata/migrate:kb-defaults
```

1.3 现有环境变量

文件	核心变量
.env (根)	SUPABASE_URL , JWT_SECRET , SUPABASE_DB_URL , MINIMAX_API_KEY , DEERAPI_*
mxmauth/.env	SUPABASE_URL , JWT_SECRET , CORS_ORIGIN , ADMIN_TOKEN
gateway/.env	PORT=3000 , JWT_SECRET , 各服务 URL , CORS_ORIGIN
mxmcgi/.env	MAXPLAN_API_KEY , BRAVE_API_KEY , MINIO_* , R2_*
mxmdata/.env	同根 .env

二、部署方案设计

2.1 设计原则

- 1. 幂等性：所有脚本可重复执行，不重复创建资源
- 2. 顺序正确：基础设施 → 数据库 → 账户 → 服务
- 3. 环境隔离：`.env.example`（模板）+ `.env`（运行时）
- 4. 快速回滚：失败时能感知并清理

2.2 部署阶段划分

- Phase 0: 环境检查 (prerequisites)
- Phase 1: 基础设施 (infra)
- Phase 2: 数据库初始化 (database)
- Phase 3: 账户初始化 (accounts)
- Phase 4: 服务启动 (services)
- Phase 5: 健康检查 (health check)

三、完整脚本实现

3.1 主入口: deploy.sh

```
#!/bin/bash
# deploy.sh - SuperMXMai 一键部署脚本

set -e # 任何命令失败立即退出
set -u # 使用未定义变量时报错

# =====
# 配置
# =====
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$SCRIPT_DIR"
DEPLOY_LOG="$PROJECT_ROOT/logs/deploy-$(date +%Y%m%d-%H%M%S).log"

# 颜色输出
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m'

log() { echo -e "${BLUE}$(date '+%H:%M:%S')${NC} $*"; }
info() { echo -e "${GREEN}[INFO]${NC} $*"; }
warn() { echo -e "${YELLOW}[WARN]${NC} $*"; }
error() { echo -e "${RED}[ERROR]${NC} $*" >&2; }

# 创建日志目录
mkdir -p "$PROJECT_ROOT/logs"

# 记录完整日志（同时输出到屏幕）
exec > >(tee "$DEPLOY_LOG") 2>&1

# =====
# 参数解析
# =====
SKIP_INFRA=false
SKIP_DB=false
SKIP_ACCOUNTS=false
SKIP_SERVICES=false
FORCE=false

usage() {
    echo "Usage: $0 [OPTIONS]"
    echo "  --skip-infra    跳过基础设施检查"
    echo "  --skip-db       跳过数据库初始化"
    echo "  --skip-accounts 跳过账户创建"
    echo "  --skip-services 跳过服务启动"
    echo "  --force         强制重新执行（跳过检测）"
    echo "  --help         显示帮助"
}

while [[ $# -gt 0 ]]; do
    case $1 in
        --skip-infra) SKIP_INFRA=true; shift ;;
        --skip-db) SKIP_DB=true; shift ;;
        --skip-accounts) SKIP_ACCOUNTS=true; shift ;;
        --skip-services) SKIP_SERVICES=true; shift ;;
        --force) FORCE=true; shift ;;
        --help) usage; exit 0 ;;
        *) error "未知参数: $1"; usage; exit 1 ;;
    esac
done

# =====
# Phase 0: 环境检查
# =====
phase0_check() {
    echo ""
    echo "=====
    echo " Phase 0: 环境检查"
    echo "=====
}
```

```

local errors=0

# 检查 Node.js
if ! command -v node &>/dev/null; then
    error "Node.js 未安装"
    errors=$((errors + 1))
else
    local node_version=$(node -v | sed 's/v//')
    local major=$(echo "$node_version" | cut -d. -f1)
    if [[ "$major" -lt 22 ]]; then
        error "Node.js 版本过低, 需要 >=22, 当前 $node_version"
        errors=$((errors + 1))
    else
        info "Node.js v$node_version ✓"
    fi
fi

# 检查 pnpm
if ! command -v pnpm &>/dev/null; then
    error "pnpm 未安装"
    errors=$((errors + 1))
else
    info "pnpm $(pnpm -v) ✓"
fi

# 检查 Docker (用于 Supabase Local + MinIO)
if ! command -v docker &>/dev/null; then
    error "Docker 未安装"
    errors=$((errors + 1))
else
    info "Docker $(docker --version | awk '{print $4}') ✓"
fi

# 检查 Docker Compose
if ! docker compose version &>/dev/null && ! docker-compose --version &>/dev
    error "Docker Compose 未安装"
    errors=$((errors + 1))
else
    info "Docker Compose ✓"
fi

# 检查环境变量文件
if [[ ! -f "$PROJECT_ROOT/.env" ]]; then
    if [[ -f "$PROJECT_ROOT/.env.example" ]]; then
        warn ".env 不存在, 复制 .env.example 为 .env"
        cp "$PROJECT_ROOT/.env.example" "$PROJECT_ROOT/.env"
    else
        error ".env 文件不存在"
        errors=$((errors + 1))
    fi
else
    info ".env 文件存在 ✓"
fi

# 检查 .env 中关键变量
if [[ -f "$PROJECT_ROOT/.env" ]]; then
    source "$PROJECT_ROOT/.env" 2>/dev/null || true
    [[ -z "${SUPABASE_DB_URL:-}" ]] && error "SUPABASE_DB_URL 未配置" && errors
    [[ -z "${JWT_SECRET:-}" ]] && error "JWT_SECRET 未配置" && errors=$((errors
fi

if [[ $errors -gt 0 ]]; then
    error "环境检查失败 ($errors 个错误), 请先修复"
    exit 1
fi

info "环境检查通过 ✓"
}

# =====
# Phase 1: 基础设施 (Docker)
# =====
phase1_infra() {
    if [[ "$SKIP_INFRA" == true ]]; then

```



```

warn "跳过基础设施部署 (--skip-infra)"
return
fi

echo ""
echo "=====
echo "   Phase 1: 启动基础设施 (Supabase + MinIO)"
echo "=====

cd "$PROJECT_ROOT"

# 检查是否已运行
if docker ps --format '{{.Names}}' | grep -q 'supabase-supabase-db-1'; then
    info "Supabase 已在运行, 跳过"
else
    info "启动 Supabase Local..."
    docker compose up -d supabase-db supabase-postgres-rest
    info "等待 Supabase 就绪..."
    sleep 10
    # 等待数据库接受连接
    wait_for_db() {
        docker exec supabase-supabase-db-1 pg_isready -U postgres -d postgres &>
    }
    local retries=30
    while ! wait_for_db && [[ $retries -gt 0 ]]; do
        sleep 1
        retries=$((retries - 1))
    done
    if [[ $retries -eq 0 ]]; then
        error "Supabase 数据库启动超时"
        exit 1
    fi
    info "Supabase 就绪 ✓"
fi

# MinIO
if docker ps --format '{{.Names}}' | grep -q 'minio'; then
    info "MinIO 已在运行, 跳过"
else
    info "启动 MinIO..."
    docker compose up -d minio
    info "MinIO 启动 ✓"
fi

info "基础设施就绪 ✓"
}

# =====
# Phase 2: 数据库初始化
# =====
phase2_database() {
    if [[ "$SKIP_DB" == true ]]; then
        warn "跳过数据库初始化 (--skip-db)"
        return
    fi

    echo ""
    echo "=====
    echo "   Phase 2: 数据库初始化"
    echo "=====

    cd "$PROJECT_ROOT"

    # 加载环境变量
    set -a && source "$PROJECT_ROOT/.env" && set +a

    # 确保 mxmdata 构建
    if [[ ! -d "mxmdata/dist" ]] || [[ "$FORCE" == true ]]; then
        info "构建 mxmdata..."
        pnpm build:mxmdata
    fi

    # 执行数据库 schema 初始化
    info "执行数据库 schema..."
    pnpm --filter @mxmai/mxmdata run init:db

```

```

# 执行数据迁移
info "执行数据迁移..."
local migrations=(
  "migrate:provider-models"
  "migrate:provider-balances"
  "migrate:provider-pricing"
  "migrate:provider-api-keys"
)

for migration in "${migrations[@]"; do
  info "执行迁移: $migration"
  pnpm --filter @mxmai/mxmdata run "$migration" 2>/dev/null || warn "迁移 $mi
done

# 设置定价 (如果提供了 setup_pricing.sql)
if [[ -f "$PROJECT_ROOT/setup_pricing.sql" ]]; then
  info "应用定价配置..."
  PGPASSWORD=${SUPABASE_DB_URL##*:} psql "${SUPABASE_DB_URL}" -f "$PROJEC
  warn "定价 SQL 执行失败, 跳过"
fi

info "数据库初始化完成 ✓"
}

# =====
# Phase 3: 账户初始化
# =====
phase3_accounts() {
  if [[ "$SKIP_ACCOUNTS" == true ]]; then
    warn "跳过账户创建 (--skip-accounts)"
    return
  fi

  echo ""
  echo "=====
  echo " Phase 3: 创建系统账户"
  echo "=====

  cd "$PROJECT_ROOT"

  set -a && source "$PROJECT_ROOT/.env" && set +a

  # 创建管理员账户
  if [[ -n "${ADMIN_EMAIL:-}" ]] && [[ -n "${ADMIN_PASSWORD:-}" ]]; then
    info "创建管理员账户..."
    pnpm --filter @mxmai/mxmdata run create:admin
  else
    warn "未配置 ADMIN_EMAIL 和 ADMIN_PASSWORD, 跳过管理员创建"
    info "可后续通过: ADMIN_EMAIL=admin@example.com ADMIN_PASSWORD=xxx pnpm --fi
  fi

  # 初始化 Prompt 模板
  info "初始化 Prompt 模板..."
  pnpm --filter @mxmai/mxmcgi run seed:prompt-config 2>/dev/null || warn "Prom

  info "账户初始化完成 ✓"
}

# =====
# Phase 4: 服务启动
# =====
phase4_services() {
  if [[ "$SKIP_SERVICES" == true ]]; then
    warn "跳过服务启动 (--skip-services)"
    return
  fi

  echo ""
  echo "=====
  echo " Phase 4: 启动服务"
  echo "=====

  cd "$PROJECT_ROOT"

```

```

# 清理占用端口
bash scripts/clear-ports.sh 2>/dev/null || true

# 启动所有后端服务
info "启动后端服务 (mxmauth, mxmcgi, mxmnotify, gateway)..."

# 使用 pnpm dev:all 启动后端
pnpm dev:all &

# 等待服务就绪
sleep 5

# 检查端口
local services=("mxmauth:4001" "gateway:3000")
for svc in "${services[@]}; do
    local name="${svc%:*}"
    local port="${svc##*:}"
    if lsof -i :"$port" -sTCP:LISTEN -P -n &>/dev/null; then
        info "$name (端口 $port) 就绪 ✓"
    else
        warn "$name (端口 $port) 未启动"
    fi
done

info "服务启动完成 ✓"
info ""
info " Gateway: http://localhost:3000"
info " 前端: http://localhost:5173"
info ""
info " 测试: curl http://localhost:3000/health"
}

# =====
# Phase 5: 健康检查
# =====
phase5_health() {
    echo ""
    echo "===== "
    echo " Phase 5: 健康检查"
    echo "===== "

    local failed=0

    check_endpoint() {
        local name=$1
        local url=$2
        if curl -sf "$url" &>/dev/null; then
            info "$name: OK ✓"
        else
            error "$name: FAIL x (无法连接 $url)"
            failed=$((failed + 1))
        fi
    }

    check_endpoint "Gateway" "http://localhost:3000/health"
    check_endpoint "mxmauth" "http://localhost:4001/health"

    if [[ $failed -eq 0 ]]; then
        info ""
        info "🎉 部署成功! "
        info ""
    else
        error ""
        error "⚠️ $failed 个检查失败, 请检查日志: $DEPLOY_LOG"
    fi
}

# =====
# 主流程
# =====
main() {
    echo ""
    echo "
    echo " SuperMXMai 部署脚本 v1.0 "
    echo " $(date '+%Y-%m-%d %H:%M:%S') "
}

```

```
echo "┌"
echo ""

phase0_check
phase1_infra
phase2_database
phase3_accounts
phase4_services
phase5_health

echo ""
info "完整日志: $DEPLOY_LOG"
}

main "$@"
```

3.2 docker-compose.yml (新增)

```
# docker-compose.yml - 基础设施服务
services:
  # PostgreSQL + PostgREST (Supabase Local Stack)
  supabase-db:
    image: supabase/postgres:15.1.0.147
    container_name: supabase-supabase-db-1
    environment:
      POSTGRES_DB: postgres
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-postgres}
      POSTGRES_HOST_AUTH_METHOD: trust
    ports:
      - "5432:5432"
    volumes:
      - supabase-db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 10

  supabase-postgres-rest:
    image: postgrest/postgrest:v12.0.2
    container_name: supabase-postgres-rest
    environment:
      PGRST_DB_URI: ${SUPABASE_DB_URL:-postgres://postgres:postgres@supabase-d
      PGRST_DB_SCHEMAS: public
      PGRST_DB_ANON_ROLE: anon
      PGRST_JWT_SECRET: ${JWT_SECRET:-}
      PGRST_DB_USE_LEGACY_GUCS: "false"
    ports:
      - "8000:3000"
    depends_on:
      supabase-db:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "wget --no-verbose --tries=1 --spider http://localho
      interval: 10s
      timeout: 5s
      retries: 5

  # MinIO (S3 兼容存储)
  minio:
    image: minio/minio:latest
    container_name: minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: ${MINIO_ACCESS_KEY:-minioadmin}
      MINIO_ROOT_PASSWORD: ${MINIO_SECRET_KEY:-minioadmin}
    ports:
      - "9000:9000"
      - "9001:9001"
    volumes:
      - minio-data:/data
    healthcheck:
      test: ["CMD", "mc", "ready", "local"]
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  supabase-db-data:
  minio-data:
```

3.3 .env.example (新增)

```
# =====
# SuperMXMai 环境变量配置
# =====
# 复制此文件为 .env 并填入实际值

# ---- 数据库 ----
# Supabase 本地开发: postgres://postgres:postgres@localhost:5432/postgres
# Supabase 云端:      postgres://postgres.xxx.supabase.co:5432/postgres?sslmode=
SUPABASE_DB_URL=postgres://postgres:postgres@localhost:5432/postgres
SUPABASE_URL=http://localhost:8000
SUPABASE_ANON_KEY=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
SUPABASE_SERVICE_KEY=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...

# ---- JWT ----
# 生成方式: node -e "console.log(require('crypto').randomBytes(32).toString('he
JWT_SECRET=your-32-char-secret-key-here-minimum
SECRET_KEY_BASE=your-64-char-secret-base-here

# ---- 管理员账户 ----
ADMIN_EMAIL=admin@example.com
ADMIN_PASSWORD=your-admin-password
ADMIN_TOKEN=your-admin-token-here

# ---- 第三方 API ----
MINIMAX_API_KEY=your-minimax-api-key
DEERAPI_BASE_URL=https://api.deerapi.com
DEERAPI_API_KEY=your-deerapi-key
MAXPLAN_API_KEY=your-maxplan-api-key
BRAVE_API_KEY=your-brave-search-key

# ---- MinIO (本地开发) ----
MINIO_ENDPOINT=localhost
MINIO_PORT=9000
MINIO_USE_SSL=false
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin
MINIO_REGION=us-east-1

# ---- R2 (可选, 生产用) ----
R2_BUCKET=mxmttemimageref
R2_PUBLIC_URL=https://pub-xxx.r2.dev
R2_ENDPOINT=xxx.r2.cloudflarestorage.com
R2_ACCESS_KEY=xxx
R2_SECRET_KEY=xxx
R2_REGION=auto

# ---- CORS ----
CORS_ORIGIN=http://localhost:3000,http://localhost:19006

# ---- 代理 (可选) ----
CLASHPROXY=http://127.0.0.1:7897
HTTPS_PROXY=http://127.0.0.1:7897
HTTP_PROXY=http://127.0.0.1:7897
```

3.4 Makefile (新增)

```
# Makefile - 常用部署命令

.PHONY: deploy deploy-infra deploy-db deploy-accounts deploy-services
.PHONY: stop clean logs status

# 默认目标: 完整部署
deploy:
    bash scripts/deploy.sh

# 分阶段部署
deploy-infra:
    bash scripts/deploy.sh --skip-db --skip-accounts --skip-services

deploy-db:
    bash scripts/deploy.sh --skip-infra --skip-accounts --skip-services

deploy-accounts:
    bash scripts/deploy.sh --skip-infra --skip-db --skip-services

deploy-services:
    bash scripts/deploy.sh --skip-infra --skip-db --skip-accounts

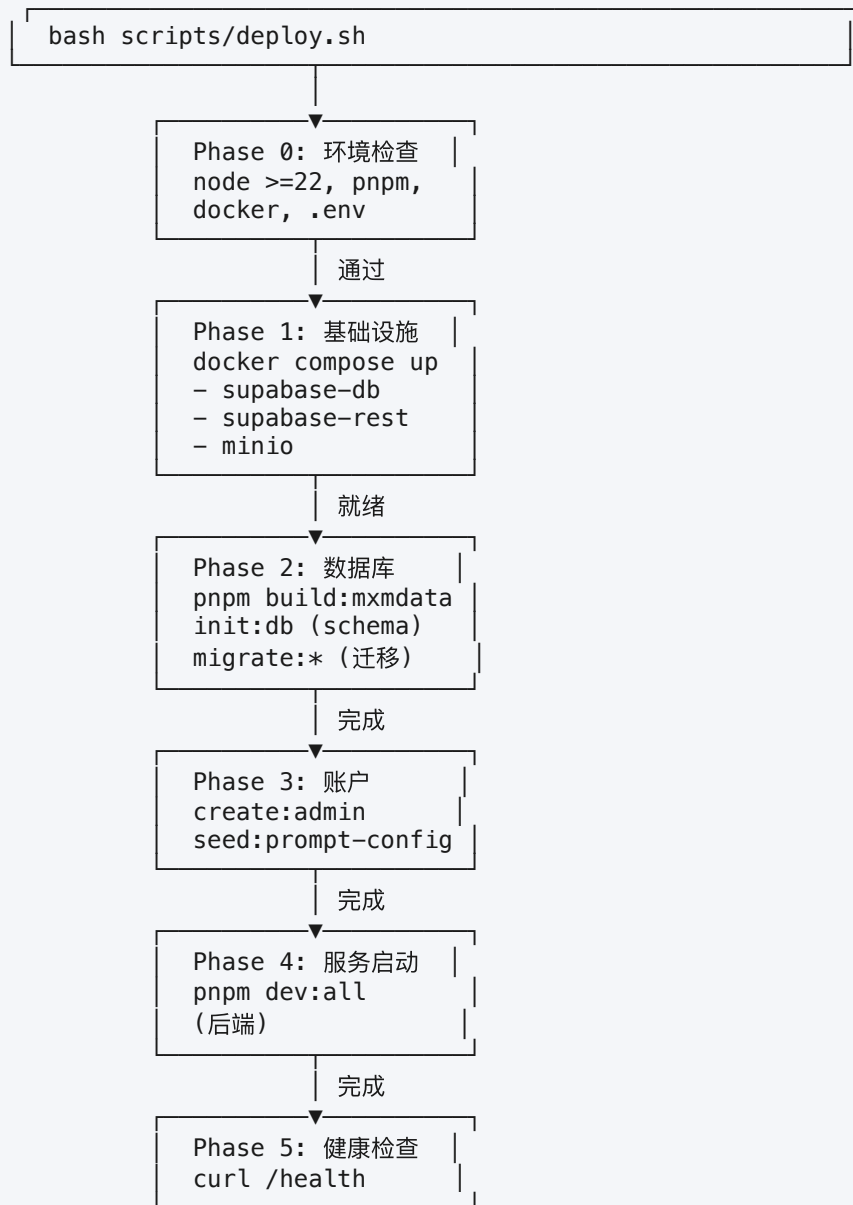
# 停止所有服务
stop:
    docker compose down
    pkill -f "pnpm.*dev" || true
    @echo "服务已停止"

# 清理
clean:
    docker compose down -v
    rm -rf node_modules/.vite
    @echo "清理完成"

# 查看日志
logs:
    @tail -f logs/deploy-$(shell ls -t logs/ | head -1)

# 状态检查
status:
    @echo "Docker 容器:"; docker ps --format "table {{.Names}}\t{{.Status}}" |
    @echo ""; echo "服务端口:"; lsof -i :3000,:4001,:4002,:4003,:4004,:4005,:5171
```

四、部署流程图



五、使用方式

5.1 首次部署

```
cd ~/Desktop/codes/supermxmai

# 1. 复制环境变量模板
cp .env.example .env
# 编辑 .env 填入实际值

# 2. 一键部署
bash scripts/deploy.sh
```


5.2 后续部署（增量）

```
# 只启动服务（数据库已初始化，跳过基础设施和数据库）
bash scripts/deploy.sh --skip-infra --skip-db --skip-accounts

# 查看状态
make status

# 查看日志
make logs
```

5.3 完整重置

```
# 清理所有数据，重新部署
make clean && bash scripts/deploy.sh --force
```

六、当前问题与改进建议

6.1 现有问题

问题	影响	改进
mxmagent 未在 dev:all 中启动	Agent 服务需手动启动	添加到 dev:all 脚本
.env.example 不存在	新环境部署繁琐	新增此文件
docker-compose.yml 不存在	无法一键启动基础设施	新增此文件
无统一入口脚本	部署步骤多、易出错	新增 deploy.sh
无 Makefile	常用命令无快捷方式	新增 Makefile

6.2 待完成项（建议 Claude Code 执行）

- 创建 .env.example
- 创建 docker-compose.yml
- 创建 scripts/deploy.sh
- 创建 Makefile
- 修改 dev:all 添加 mxmagent
- 修改 start-services.sh 为 Shell 风格（兼容 bash）
- 添加 rollback 脚本（失败时清理）
- 添加 CI/CD 部署流水线（GitHub Actions）